SAP BRIM AI Architect — Reference Answers

10 Oct 2026 · @DS

How to use these answers

These are draft reference answers (about 200 words each) for the 92 questions written out in the blueprint, plus the BRIM-ARCH-001 flagship scenario. They are a starting baseline for grading the agent, not approved solutions: each one must be reviewed and signed off by two experienced BRIM consultants before it is stored as a reference answer.

- **Release sensitivity.** Object names (charge plan, provider contract, billable item class, ODI) are SAP standard concepts. Transaction codes, Fiori app names and IMG paths change between S/4HANA, SAP CC and CM releases; any code named here is marked *verify* and must be confirmed in the client's system before use.
- **Standard first.** Every answer separates SAP standard capability from proposed custom extensions.
- **Not covered.** The blueprint lists eight migration questions (MIG-01 to MIG-08) but does not include their text, so they are not answered here. Send the question text and they can be added.
- **Production safety.** No answer authorises direct table updates, unchecked replays or financial reversals without approval gates.

Answer contract used: requirement and assumptions → module ownership → configuration and sequence → data and samples → testing → monitoring and recovery → risks and recommendation.

SOM — Subscription Order Management

SOM-01 — Complete SOM configuration

SOM in S/4HANA owns the commercial lifecycle: subscription order, subscription contract and change processes; it distributes provider contracts to SAP CC and FI-CA/CI. Configure in this sequence. (1) Organisational basis: sales organisation, distribution channel, division, company code and FI-CA contract account category. (2) Business partner: BP roles for customer and FI-CA contract partner, contract account with billing and dunning parameters. (3) Product model: subscription product with product type, billing-relevant settings, rate plan reference and technical resource IDs. (4) Transaction types and item categories for subscription order and subscription contract, with item category determination. (5) Change processes (change, renew, suspend, terminate) assigned to contract item categories. (6) ODI (Order Distribution Infrastructure) distribution categories and step sequences for provider contract

creation in FI-CA and CC. (7) Billing integration: billing cycle, billable item class mapping in CI.

Master data example: BP 1000123, contract account 900045, product SUB-PREMIUM with rate plan RP-PREM-M. Transport strategy: separate Customizing (IMG), CC catalogue (via CC Cockpit export) and master data; never transport BP data. SIT cases: order to active contract, provider contract present in FI-CA and CC, first recurring charge billed, change and termination flows. Verify every IMG path against the client release.

SOM-02 — Product and rate-plan modelling

Model one subscription product with three charge components, and keep pricing ownership explicit. In SOM, the product carries the rate plan reference and parameters (plan tier, activation indicator); SOM does not calculate the usage price. In SAP CC, the charge plan contains: a recurring charge (₹25,000 monthly, triggered by the recurring event on the billing cycle), a one-time charge (₹50,000 activation, triggered once on contract activation) and a usage charge (rated per chargeable item from mediation). Charge plan parameters received from the provider contract select tiers or discounts.

The contract item references the product; at activation, ODI creates a provider contract carrying the charge plan ID and parameter values. CC outputs charged items (or consumption items, depending on release) that become billable items in CI, classified by billable item class and subprocess. CI groups and invoices; FI-CA posts receivables.

Ownership: SOM = what was sold and when; CC = how much it costs; CI = what goes on which invoice; FI-CA = what the customer owes. Test: activation creates exactly one ₹50,000 billable item, the first cycle creates one ₹25,000 item (prorated if mid-cycle per the agreed policy), and usage items equal rated quantity × tier price.

SOM-03 — Contract distribution failure

When a SOM contract is active but no provider contract exists in SAP CC, the break sits in ODI distribution or the CC interface. Architecture: contract activation triggers an ODI distribution document; its step sequence creates the FI-CA provider contract and then sends the provider contract to CC through the S/4HANA–CC connection (RFC/web service, release dependent). Investigation: (1) Open the ODI monitor (app or transaction — verify for release) and locate the distribution document by contract number. (2) Check step status: not started (scheduling), error (read message log) or waiting on a predecessor step. (3) Confirm the FI-CA provider contract exists and is consistent. (4) In CC, check whether the subscriber account and charge plan referenced exist and whether the inbound call logged an error (missing charge plan, invalid parameter, connection timeout). (5) Check the communication user and destination.

Reprocessing: fix the root cause (catalogue, master data, destination), then restart the failed ODI step from the monitor; never create the CC contract manually as a production fix, because it breaks identifier alignment. Permanent actions: alerting on ODI errors older than a threshold, a daily SOM-versus-CC contract count reconciliation, and catalogue release checks before products go live.

SOM-04 — Contract change processes

Configure each change as a standard change process assigned to the contract item category, with its own validity and pricing logic. Upgrade and downgrade replace the product or rate plan from an effective date and create a new time slice; renewal extends the contract end date and can re-price; suspension and resumption set item status and suspend recurring charges; cancellation (before start) and termination (after start, with notice periods and possible early termination fee) end the item.

Effective dates drive everything: SOM creates a change order, the contract gets a new time slice, and ODI distributes a provider contract change to FI-CA and CC with the same effective date. CC applies the new charge plan from that date, so recurring charges are prorated per the charge plan's proration rule and usage after the date rates on the new plan.

Billing implications: billing must not run for a period while its change is still in distribution. Regression test set: each change on the 1st, mid-cycle and last day of cycle; future-dated change; two consecutive changes; change after invoice; termination with fee; suspension spanning a billing run. Expected result for each: correct time slices in SOM, CC contract version and billable item amounts.

SOM-05 — Master agreement design

Use the SOM master agreement as the commercial umbrella and individual subscription contracts per subsidiary underneath. The master agreement holds negotiated conditions: authorised products, price agreements or discounts and validity. Each subsidiary gets its own business partner (sold-to) linked to the corporate parent through a BP hierarchy or relationship, plus its own contract account so billing arrangements and payment terms can differ.

For 200 subsidiaries with different discounts, choose one of two patterns: (a) one master agreement with subsidiary-specific conditions where the release supports partner-dependent agreement conditions, or (b) a parent master agreement with child agreements per region or discount band. Discounts that affect rating should be passed as charge plan parameters to CC, not hard-coded per customer in the catalogue.

Limitations to validate: number of conditions per agreement, how agreement changes propagate to existing contracts (usually not automatic), and reporting across the hierarchy. Governance: a single owner per agreement, approval workflow for condition

changes, validity end-date monitoring and a quarterly reconciliation of active contracts against agreement terms. Invoice consolidation across subsidiaries is a CI and contract account decision, not an agreement setting.

SOM-06 — Bundled offerings

Model the bundle as a sales package with component items, each with its own item category and fulfilment path. Equipment is a physical sales item (delivery, possibly serial numbers, billed through SD or CI one-off); software licence and maintenance are subscription items (recurring charges in CC); installation is a one-time service item; consumption is a usage charge on the subscription item's charge plan.

Contract creation: the bundle order creates one subscription contract with linked items; ODI creates the provider contract for the subscription items, while equipment and installation follow logistics and service order processes. Invoicing: all components become billable items with a common reference (contract or bundle ID) so CI can group them on one invoice; equipment billing from SD can be integrated into CI where the release supports it.

Revenue recognition: a bundle has multiple performance obligations, so allocate the transaction price by standalone selling price in SAP RAR (or the revenue solution in use). Billing amounts are not revenue amounts. Key controls: bundle discount allocation rule, link between items for cancellation (can maintenance be cancelled without the licence?), and testing of partial fulfilment, early termination and swap of one component.

SOM-07 — Mass subscription changes

Use the SOM mass change capability (or a mass change program built on standard change process APIs) rather than editing provider contracts. Design: (1) Eligibility selection — active contracts with the affected products, excluding contracts with price locks, pending changes or termination dates before 1 January. (2) Change process: price change effective 1 January creating a new time slice; history stays untouched because earlier time slices and already-rated items are not modified. (3) Where the price lives in CC, a 7% catalogue change (new charge plan version or price table validity from 1 January) may be simpler than 500,000 contract changes; choose this if no contract-specific prices exist.

Scheduling: run in packages (for example 10,000 contracts), in parallel within limits, well before billing of January, with ODI capacity tested. Performance controls: lock handling, throttling of ODI distribution, monitoring of queue depth. Recovery: idempotent job design (a contract already changed is skipped), error list with restart per package. Reconciliation: count of eligible contracts = changed + excluded + failed; CC

contract versions match; a sample of January invoices shows exactly 7% uplift with correct rounding.

SOM-08 — Backdated amendments

A 45-day backdated upgrade after two invoices is a retroactive change; handle it through a standard change process with a past effective date, if the release and business rules allow it. Steps: (1) Approval — retro changes need sign-off because they alter billed periods. (2) Create the change with the past effective date; SOM creates the corrected time slice and ODI distributes the provider contract change. (3) In CC, the new charge plan applies from the effective date; recurring charges for the affected period must be recalculated, and usage in the window must be rerated.

Adjustment billing: the delta should appear as reversal or credit of the original charges and new charges at the upgraded price, producing billable items for the next invoice (or a correction invoice). Never delete the original billable items; keep the audit trail. Financial reconciliation: original invoices plus adjustment equals what the customer would have paid if the upgrade had been in place from day one; FI-CA open items and tax recalculate on the adjustment.

Design rule: set a maximum retro window (for example 60 days) by policy and handle anything older through manual credit memo with approval.

SOM-09 — Multi-country subscriptions

Model each country's legal entity as its own company code and sales organisation, with FI-CA contract accounts per legal entity. The global customer becomes one business partner (or a parent with local child BPs where local tax IDs and addresses differ), and each local subscription is a contract under the local sales organisation.

Currency: the contract currency is the local selling currency; CC rates in that currency or converts by an agreed rule; CI invoices in the contract account currency. Tax: tax determination happens at invoicing using the company code, ship-to and product tax classification, through standard tax or an external tax engine. Invoice formats: output determination per company code and country.

Integration: CI and FI-CA post per company code; cross-company consolidated views come from reporting, not from one invoice crossing legal entities, because a legal invoice cannot combine two issuing entities. Global master agreement terms can sit above the local contracts.

Test per country: order to invoice, local tax correctness, currency rounding, e-invoicing format where mandated (India e-invoice, EU rules) and dunning in local currency. Open decision to confirm: whether intercompany billing is required where one entity delivers services sold by another.

SOM-10 — Contract lifecycle automation

Standard SOM supports renewal processes, contract end dates and auto-renewal settings; notification, approval and dynamic pricing usually need extensions. Design: (1) A scheduled job selects contracts ending in 90 days and creates a renewal task or triggers a workflow. (2) A notification goes to the customer through an output channel or CRM (Salesforce or SAP Sales Cloud). (3) Customer approval returns through an API; the renewal change process runs with new end date and price. (4) Without a response, the contract follows its rule: auto-renew, renew at list price, or terminate at end date.

Renewal price: calculate with the pricing rule (for example, list price plus uplift, capped) in SOM or as a CC catalogue version; keep the calculation in one place. Failure monitoring: dashboard of contracts within 30 days of expiry without a decision, renewal orders in error and ODI distribution failures.

Extensions needed: notification scheduling logic, approval capture, and possibly a BTP workflow. Keep them clean-core: use released APIs and event-based triggers rather than modifications. Tests: approve, reject, no response, price recalculation, renewal of a contract with a pending change, and renewal at the exact end date.

SOM-11 — ODI architecture

ODI is configured as distribution categories (which kind of document is distributed), schema determination (which step sequence applies, based on transaction type, item category or process) and step types (the actual actions: create FI-CA provider contract, send to CC, notify an external system). Steps have dependencies: CC distribution normally waits until the FI-CA provider contract exists, because CC needs the contract account and provider contract identifiers.

Example with two activations. Immediate activation: the order completes, the distribution document is created at once, step 1 creates the FI-CA provider contract, step 2 sends the provider contract to CC, step 3 updates SOM status to distributed. Future-dated activation (start 1 March, sold 10 February): the distribution can happen immediately with a future valid-from date in FI-CA and CC, or the step can be scheduled for the start date. Distributing early is usually safer, since errors surface before the service starts.

Scheduling: background job for pending and retry steps at a defined interval. Recovery: steps are restartable; errors are visible in the ODI monitor with message logs; configure automatic retries only for technical errors (timeouts), never for business errors (missing charge plan). Verify configuration activity names and the monitor app in the client's release.

SOM-12 — SOM interface design with Salesforce and CC

Set clear systems of record: Salesforce owns the opportunity, quote and customer-facing order capture; SOM owns the subscription contract and its lifecycle; CC owns rating and charging state. Salesforce never writes to CC directly.

Flow: Salesforce sends an order to SOM through a released S/4HANA API, ideally via SAP Integration Suite. SOM validates, creates the subscription order and contract, and ODI distributes to FI-CA and CC. SOM returns the contract number and status to Salesforce asynchronously (event or callback).

Identifiers: keep the Salesforce order ID as an external reference on the SOM order; the SOM contract ID is the master key; the CC provider contract ID is derived from SOM. Duplicate protection: an idempotency key (Salesforce order ID plus version) checked in Integration Suite or in SOM before creation, so retries cannot create a second contract.

Retry policy: technical failures retry with exponential back-off and a dead-letter queue; business validation failures return an error to Salesforce for user correction. Security: OAuth 2.0 client credentials or certificates, a technical user with least-privilege roles, TLS everywhere, and payload logging that masks personal data. Monitoring: end-to-end message tracking by order ID across Salesforce, Integration Suite and SOM.

SOM-13 — Contract inconsistency between SOM and CC

If SOM shows Active but the CC contract has outdated parameters, a change was recorded in SOM but not applied downstream. Diagnostic sequence: (1) List SOM contract change history and time slices; note the last change and effective date. (2) In the ODI monitor, check whether that change created a distribution document and its step status. (3) Compare the FI-CA provider contract version with SOM. (4) Compare the CC provider contract: charge plan, parameter values and validity periods per time slice.

Likely causes: distribution step failed or is waiting; a change process not configured to trigger distribution for that parameter; a mapping gap (the SOM attribute is not mapped to a CC parameter); a manual change in CC that was later overwritten; or concurrency, where a second change was distributed before the first finished.

Data comparison strategy: extract both sides keyed by contract ID and time slice and compare field-by-field for the affected population, not just the one complaint. Safe correction: fix the cause, then redistribute from SOM through ODI so SOM remains the source of truth. Before correcting, identify charges already rated on the wrong parameters; those need rerating and billing adjustment under approval.

SOM-14 — High-volume performance (one million subscriptions per month)

One million per month is about 33,000 per day on average, but plan for peak, for example 10× during campaigns or migration waves. Design: asynchronous order intake through Integration Suite queues so the front end never waits for distribution; SOM

order processing in parallel packages; ODI distribution run by parallel background jobs with a controlled number of work processes.

Throughput assumptions to measure, not guess: orders per hour that SOM creates, ODI steps per hour, and CC contract creations per second. Set targets such as "peak day of 150,000 activations completed within 8 hours" and test them.

Locking risks: multiple orders for one business partner or one contract account updating the same objects at once. Mitigate by grouping orders by business partner in the same package, or by serialising changes per contract. Monitoring: queue depth, ODI backlog age, error rate, CC response time and dialog/background work process usage.

Performance tests: volume test at expected peak, endurance test over several days, and a recovery test where CC is unavailable for one hour and the backlog drains afterwards. Size the HANA database, application servers and CC servers with SAP sizing guidance and the measured results.

SOM-15 — Architecture decision: standard change process versus custom API

Recommendation: use standard SOM change processes, extended through released APIs where needed, and build a custom orchestration layer only for the bulk logic, not a custom change engine.

Criterion	Standard change processes	Custom change API
Upgrade compatibility	High; maintained by SAP	Low to medium; regression on every upgrade
Build cost	Low configuration effort	High development and testing
Scalability	Proven, needs packaging for bulk	Can be tuned, but you own it
Auditability	Full change history and time slices	Must be designed; risk of gaps
Downstream distribution	ODI triggered automatically	Must replicate ODI triggering
Operational risk	Low	High; inconsistency between SOM and CC

The main risk of a custom change API is bypassing the standard change history and distribution, which creates SOM-versus-CC drift. A safe hybrid: a bulk orchestration program or BTP service that selects contracts, applies business rules and calls the

standard change process API per contract in packages, with idempotency and restart logic. This keeps SAP data ownership intact and limits custom code to orchestration.

SOM-16 — Functional design document outline

An implementation-ready FDD for activation, recurring billing, mid-cycle upgrade, renewal and termination contains these sections. (1) Scope and business requirements with IDs. (2) Process flows for each lifecycle event, from order to invoice. (3) Business rules: activation date rule, proration rule (daily, by calendar days), upgrade effective date, renewal pricing, notice period and early termination fee. (4) Data model and mappings: SOM product and item attributes to CC charge plan parameters, contract account and billing cycle to CI, external references from CRM.

(5) Interfaces: CRM to SOM, SOM to FI-CA and CC (ODI), CC to CI, with message types and error handling. (6) Exception handling: distribution failure, rating failure, billing block, retry and owner of each queue. (7) Authorisations and segregation of duties. (8) Reporting and reconciliation: contract counts, billed amount versus expected. (9) Assumptions and dependencies: release, catalogue readiness, master data quality. (10) Acceptance criteria per requirement, such as "a mid-cycle upgrade on day 16 of a 30-day cycle produces a 15-day credit at old price and a 15-day charge at new price". (11) Open issues and sign-off table. Keep the FDD traceable to test cases in a requirements traceability matrix.

SOM-17 — Configuration workbook

Use one row per configuration object, with these columns: ID, area (SOM, ODI, FI-CA, CI, CC), configuration object or IMG activity, value or key, purpose, dependency (prerequisite IDs), responsible consultant, DEV status, QA validation date, PRD validation date, test case ID and evidence link, and transport request number.

ID	Area	Object	Purpose	Depends on
C-001	FI-CA	Contract account category	Billing-relevant accounts	Company code
C-010	SOM	Subscription contract transaction type	Contract header control	C-001
C-011	SOM	Item category and determination	Item behaviour	C-010
C-020	SOM	Change process assignment	Upgrade, renew, terminate	C-011

C-030	ODI	Distribution category and schema	Provider contract distribution	C-011
C-040	CI	Billable item class mapping	Accept CC output	CC catalogue

Rules: dependencies drive transport sequence; every row has evidence before QA sign-off; CC catalogue objects are tracked in the same workbook even though they are transported through the CC export or import mechanism. The workbook becomes the baseline for upgrade impact analysis later.

SOM-18 — Defect: change orders fail while a previous change is distributing

The pattern (30% failure only when a prior change is still in distribution) points to a sequencing and locking problem: the second change tries to update a contract or provider contract that is locked, or that does not yet reflect the first change. Diagnose by correlating failed orders with ODI timestamps: for each failure, check whether an earlier distribution document for the same contract was open. Read the error messages, which typically show a lock or version conflict.

Standard-first options: (1) Configure SOM so a new change cannot be released while the contract has an open distribution; the user or API receives a clear message. (2) Use ODI dependency or serialisation so distributions for the same contract run in order. (3) Shorten distribution latency by scheduling the ODI job more frequently, reducing the window.

Recovery: re-run failed change orders after the earlier distribution completes, in the original business order, and validate CC contract versions. Prevention: test cases for back-to-back changes; integration layer that queues changes per contract; monitoring of contracts with more than one change in flight. Avoid custom lock-bypassing code, because it creates silent inconsistency.

SOM-19 — UAT scenarios (summary of 50)

Organise the 50 UAT scenarios in eight groups, each with preconditions, steps and expected results across SOM, CC, CI and FI-CA.

1. Creation (8): new customer, existing customer, future start, bundle, multi-item, foreign currency, master agreement price, missing data (negative).
2. Modification (8): upgrade and downgrade on day 1, mid-cycle and last day; parameter change; quantity change; add-on item.
3. Suspension and resumption (5): within a cycle, across a billing run, with usage during suspension (expected rejection or zero rating).

4. Termination (7): at end, early with fee, backdated, cancellation before start, termination with open usage.
5. Date changes (5): start date moved forward and back, end date extension, renewal.
6. Overlapping amendments (6): two changes same day, change during pending distribution, change and termination together.
7. Failed distribution (5): CC unavailable, missing charge plan, retry, manual restart, reconciliation.
8. Downstream billing (6): first invoice, prorated invoice, consolidated invoice, credit on downgrade, tax, FI-CA posting.

Each scenario records business-user sign-off and evidence (screenshots or reports of contract, CC contract, billable items, invoice).

SOM-20 — Architecture review: modifying provider contracts directly

The proposed design breaks the core rule that SOM is the system of record for the subscription contract. Changes made directly on downstream provider contracts (in FI-CA or CC) are not reflected in SOM, so SOM shows one commercial state and charging uses another.

Violations: (1) Data ownership — two masters for contract terms. (2) Audit risk — the change history in SOM does not show what the customer is charged for; auditors cannot trace price changes to an approved order. (3) Lifecycle risk — the next standard SOM change redistributes SOM data and silently overwrites the manual change, or fails on version mismatch. (4) Reporting, renewal and revenue accounting read SOM, so they are wrong. (5) Segregation of duties is weakened because charging-side users can alter commercial terms.

Safer design: all modifications go through SOM change processes (UI, API or mass change), and ODI distributes them. Lock down direct provider contract changes with authorisations, allowing only an emergency procedure with approval, logging and mandatory follow-up correction in SOM. Add a periodic SOM-versus-CC consistency report to detect drift.

CC — Convergent Charging

CC-01 — Full Convergent Charging implementation

SAP CC calculates prices for recurring, one-time and usage events and hands charged results to CI. Design and configure in this order. (1) Landscape: CC Core Server, database, and the administration tool for the release (CC Cockpit in recent releases; Core Tool in older ones — verify). (2) Catalogue: service providers, then charges (the

pricing logic for one charge type: recurring, one-time or usage), charge plans that bundle charges, and parameters exposed for contract-specific values. (3) Reference data: mapping tables (input attributes to output values), range tables (tiers and thresholds), and translation tables. (4) Subscriber data: subscriber accounts and provider contracts, normally created by SOM through ODI, not manually. (5) Integration: inbound from SOM (contract), inbound usage from CM (chargeable items), outbound charged items or consumption items to CI through the configured bulk loader or connector.

Validation per object: simulate each charge with test chargeable items in the charge simulation function; test the charge plan against a provider contract in QA; confirm the output format matches the CI billable item class fields. End-to-end check: one contract with recurring, one-time and usage charges produces the expected billable items in CI with correct amounts, quantities and references.

CC-02 — Tiered usage pricing

Model the pricing in one usage charge using a range table on a monthly counter of cumulative units, with a counter reset per billing period. Tiers: 0–1,000 units at ₹0; 1,001–10,000 at ₹2; above 10,000 at ₹1.50. Because pricing is tiered (graduated), each unit is priced in the band it falls into; the charge logic must split a chargeable item that crosses a threshold.

Monthly units	Calculation	Amount
500	500 × ₹0	₹0
5,000	1,000 × 0 + 4,000 × 2	₹8,000
10,000	1,000 × 0 + 9,000 × 2	₹18,000
15,000	1,000 × 0 + 9,000 × 2 + 5,000 × 1.50	₹25,500

Tiered versus volume: under volume pricing, the tier reached prices all units. At 15,000 units that would be 15,000 × ₹1.50 = ₹22,500, which creates a price drop when crossing 10,000 (10,000 units = ₹20,000 versus 10,001 units ≈ ₹15,001.50). Volume pricing therefore suits end-of-period batch rating, while tiered pricing works naturally with event-by-event rating. Tests must include the threshold units 1,000, 1,001, 10,000 and 10,001, a single event crossing a threshold, and the reset on period change.

CC-03 — Recurring and one-time charges in one subscription

One charge plan holds four charges. The recurring charge (monthly fee) is triggered by the recurring event of the provider contract per the billing cycle; proration on start, change and end uses the charge's proration setting (by days in period is common). The activation charge is a one-time charge triggered at contract activation. The cancellation

fee is a one-time charge triggered by the termination event, conditional on termination within the minimum term (passed as a parameter or derived from the contract dates). The usage charge rates chargeable items from mediation.

Triggers depend on SOM sending the right lifecycle events through the provider contract: activation date, change dates and termination date. CC creates charged items with an indicator for charge type so CI can map them to different billable item types and subprocesses (for example, recurring and usage on different invoice lines).

Billing interface: charged items flow to CI, where billing cycle alignment matters; the recurring charge period should match the CI billing period to avoid partial periods across invoices. Test: activation on the 1st and 16th, termination inside and outside the minimum term, and usage on the day of termination.

CC-04 — Mapping tables for multi-dimensional pricing

Use a mapping table class with four input columns (country, customer segment, service category, contract tier) and output columns (unit price, currency, discount percentage). Inputs come from chargeable item fields (service category) and from provider contract parameters (country, segment, tier). The charge looks up the mapping table, then applies the output price.

Design: keep the table normalised; prefer wildcards or default rows for combinations not listed, so a new segment does not cause rating failure. Define a fallback rule: if no row matches, either use a default price row or reject the item to a suspense queue for investigation; for financial accuracy, rejection with alerting is usually safer than a silent default.

Validity management: mapping table rows carry validity periods, so a price change becomes a new row valid from a date, while history keeps the old price for back-rated usage.

Deployment: maintain mapping table content in DEV, test in QA, then transport through the CC catalogue export or import or the release's change management; for frequently changing business data, decide whether business users maintain it directly in production under approval. Test every dimension at its edges plus a no-match case.

CC-05 — Charging contract integration failure

When SOM released the contract but CC lacks the expected charge-plan parameters, follow the data path. (1) SOM: confirm the contract item carries the parameter values, and that the change or activation created a distribution document. (2) ODI: check the distribution step status and message log; a successful step means CC accepted a payload. (3) FI-CA: confirm the provider contract exists with the correct identifiers. (4) CC: look up the provider contract and subscriber account; compare the charge plan and parameter values per validity period.

If the CC contract exists but parameters are blank or default, the cause is usually mapping: the SOM attribute is not mapped to the CC charge plan parameter name, the parameter name in the catalogue changed, or the parameter is optional and the default was taken. If the contract is missing, the issue is distribution or connection (see SOM-03).

Subscriber mapping: confirm the subscriber account in CC links to the right contract account; a wrong link misroutes charged items. Correction: fix the mapping, then redistribute from SOM. Check items already rated with the wrong parameters, quantify the financial impact, and rerate under an approved correction process.

CC-06 — Volume discounting

Model volume discounts with counters in the charge plan. A counter accumulates monthly volume (units or amount) per provider contract or per shared scope; a range table maps counter value plus customer category plus product to a discount rate. Customer category and product come from provider contract parameters.

Counter design: define the reset period aligned with the billing cycle; decide whether the discount applies incrementally (tiered: only volume above the threshold gets the discount) or retroactively to all volume once the threshold is reached. Retroactive ("all units") discounts cannot be finalised in real time, because early events were priced before the threshold was known. Two options: rate at list price and apply a period-end discount through a discount charge or a CI-level adjustment, or rerate the period at month end.

Retroactive volume changes (late usage increases the month's volume): if pricing is tiered, late events are rated at the current counter state, which can differ from on-time rating; document this. If volume is all-units, a period-end recalculation handles late events naturally until the period closes, after which adjustments go through rerating.

Tests: crossing each threshold, counter reset, late event after reset, and category change mid-month.

CC-07 — Shared allowances across five subsidiaries

The requirement is a shared counter (100,000 units per month) consumed by five provider contracts. Options depend on CC capabilities in the release: (a) a shared or group-level counter on a parent account or sharing contract, referenced by the subsidiaries' contracts, if supported; (b) route all usage to one provider contract on the parent, with the subsidiary as an attribute for invoice breakdown; or (c) rate at list price and apply the allowance at period end through a custom calculation. Validate which pattern the CC release supports before design sign-off.

Concurrency: with real-time rating, simultaneous events from different subsidiaries update the same counter; CC serialises updates on the counter, which can become a throughput bottleneck at high event rates. Batch rating reduces contention.

Overage: after 100,000 units, overage price applies; decide who bears the overage (the subsidiary whose event crossed the threshold, or proportional allocation at period end). Allocation by consumption share is fairer but needs period-end calculation.

Tests: allowance exhausted by one subsidiary, by all five evenly, concurrent events at the threshold, late usage after exhaustion, and a subsidiary leaving the group mid-month.

CC-08 — Multi-currency pricing

Decide where currency is set: the provider contract carries the charging currency, consistent with the FI-CA contract account currency used by CI. Best practice is to price in the invoice currency directly (USD, EUR, INR and AED price rows in mapping or range tables), so no conversion happens in rating.

If prices exist only in one base currency, conversion is needed; own it in one place. Options: convert in CC using an exchange-rate table maintained from the S/4HANA rate type, or price in base currency and convert in CI or FI-CA. Converting in CC fixes the rate at rating time; converting at invoicing fixes it at billing date. Choose based on the commercial contract.

Rounding: define decimal precision per currency (INR 2, AED 2, plus any CC internal precision), round at the charged item or invoice line as agreed, and document the rule because rounding differences show up in reconciliation.

Rate validity: exchange rates have validity dates; late usage must use the rate valid at the event date or at the rating date — state which. Reconciliation with CI: compare totals per currency for charged items versus billable items; differences should only come from rounding within tolerance.

CC-09 — Backdated usage (60 days late)

Rate the late usage against the pricing valid on the event date. CC catalogue objects and provider contract versions are time-dependent, so an event with a consumption date 60 days back is priced with the charge plan version and parameters that applied then, provided the contract and catalogue history still exist for that date.

Checks before rating: (1) Is the event within the accepted late-usage window? Set a policy, for example 90 days. (2) Was the contract active on the event date? (3) Does the counter for that past period still exist, and should the event affect it? Tiered counters for closed periods usually should not be reopened silently.

Billing: the charged item flows to CI as a new billable item with the original event date; CI bills it in the next billing run as a separate line referencing the earlier period, because the original invoice is already posted. Mark these items with a late-usage indicator so the invoice shows them clearly and reporting can track them.

Controls: approval if the late amount exceeds a threshold, customer communication, and tax using the rules valid on the supply date. Never reopen or modify the posted invoice.

CC-10 — Rerating 200,000 incorrect records

Treat rerating as a controlled financial correction. (1) Containment: fix the catalogue error with a new version and stop further wrong rating. (2) Scope: identify affected charged items by charge, validity window and contract; split into not-yet-billed and already-invoiced. (3) Not-yet-billed items: reverse them in CC or remove them from CI before billing using the standard reversal mechanism, then rerate the original chargeable items. (4) Already-invoiced items: rerating produces reversals of the original amounts and new amounts; CI bills the difference as credit or debit lines on the next invoice or a correction invoice.

Preventing duplicates: rerate from the stored original chargeable items using their unique IDs, and ensure the reversal of each original charged item happens exactly once; reconcile counts (200,000 reversed, 200,000 re-charged). Use the standard rerating capability of the release where it exists; avoid deleting records.

Evidence: before and after amounts per contract, total financial impact, approval record from finance, and a sample of customer invoices checked manually. Reconciliation: original total + reversals + new charges = corrected expected total. Communicate with affected customers before large credits or debits appear.

CC-11 — High-volume rating architecture (50 million events per day)

50 million events per day averages about 580 per second; plan for a peak of three to five times, so 2,000 to 3,000 per second. Workload assumptions to confirm: event size, share of real-time versus batch, number of charges per event, counter usage and number of active contracts.

Architecture: CM aggregates where business rules allow (reducing events reaching CC), then sends chargeable items in bulk to CC; CC Core Server runs as a scaled cluster with multiple dispatcher and updater instances as supported by the release; the database is sized for the write load of charged items and counters.

Parallelism: distribute load by subscriber or contract so updates on the same counter stay on one processing path, avoiding lock contention. Back-pressure: CM must buffer and throttle when CC slows, using persistent queues rather than dropping or retrying blindly.

Failure recovery: every chargeable item has a unique ID; CC rejects duplicates; CM resends only unacknowledged items. Sizing inputs: events per second at peak, contracts, charges per event and retention period of charged items. Validate with SAP sizing guidance and a load test at 150% of peak.

CC-12 — Charging failures with timeouts at peak

With 15% of items failing on timeout, separate the components. (1) Interface: measure latency between CM and CC; check connection pool size, network latency and whether CM opens too many parallel sessions. (2) CC processing: check CC server CPU, memory and thread usage; look for queue build-up and slow charges. (3) Database: check write latency, lock waits on counters and storage throughput. (4) Configuration complexity: charges with many mapping table lookups or heavy counter logic increase per-item time; identify the slowest charges in the CC performance statistics.

Retry handling: timeouts do not prove the item failed; the item may have been charged and the acknowledgement lost. Retries must use the same chargeable item ID so CC deduplicates; otherwise retries cause duplicate charges. Set a retry limit with exponential back-off and push items still failing to a suspense queue.

Immediate containment: reduce CM parallelism to a sustainable rate, or move part of the load to batch rating. Root-cause actions: tune connection pools, add CC capacity, simplify slow charges, and run a load test at peak to confirm. Reconciliation after the incident: sent = charged + rejected + suspended.

CC-13 — Charge-plan versioning

CC catalogue objects support time-dependent versions: create a new version of the charge or mapping table rows with validity from the first day of next month, leaving the current version valid until the day before. Usage is rated by the event date, so historical usage, including late usage, keeps its original price.

Future-dated contracts: a contract signed today starting next month picks the new version automatically. Contracts with price locks need either a separate charge plan or parameters that preserve their price.

Regression tests: an event dated the last day of the old version and the first day of the new one; a late event for last month after go-live; a recurring charge for a period that spans the change (if the price changes mid-period, check proration); simulation of every changed charge.

Rollback: because the old version remains, rollback means ending the new version (setting its validity to start later or deactivating it) before any event has been rated with it. If events were already rated on the new version, rollback requires rerating. Promote catalogue versions through QA with sign-off and keep the export of the previous catalogue as a backup.

CC-14 — Duplicate charging from mediation retries

Deduplication belongs at two layers, with CM as the primary control and CC as a safety net. In CM, each usage record gets a stable unique event ID (source system plus source record ID plus timestamp, or a hash of these) before any processing; CM stores processed IDs for a retention window and drops duplicates. In CC, chargeable items carry a unique ID, and the release's duplicate check rejects an item already charged; confirm how this is configured for the release.

Retry design: the CM-to-CC transfer must be idempotent; a resend after a timeout uses the same chargeable item ID, so CC answers "already processed" instead of charging again. Never generate a new ID on retry.

Responsibility: the source platform is responsible for a stable record ID; CM for detecting source-level duplicates; CC for rejecting transfer-level duplicates. CI should not be relied on to remove duplicates, because by then the financial amount already exists.

Tests: the same record sent three times, the same record with a changed quantity (a correction, not a duplicate), duplicates across file boundaries, and duplicates after a CM restart. Monitor the duplicate count per source as an operational KPI.

CC-15 — Real-time versus batch rating

For 20 million daily events with balance visibility and overnight billing, recommend a hybrid. Real-time (online) rating gives an immediate price and balance, needed for prepaid scenarios, spending limits or live customer dashboards; it costs more capacity and makes counters a contention point. Batch (offline) rating processes files or bulk transfers efficiently and suits postpaid billing.

Decision based on measurable requirements: (1) Balance visibility latency — if the business accepts 15 minutes, near-real-time micro-batches are enough. (2) Spending control — if service must be blocked at a limit, real-time rating is required for those customers. (3) Billing deadline — overnight billing needs all events rated by a cut-off, achievable in batch.

Recommended architecture: CM collects continuously and sends micro-batches to CC every few minutes (about 230 events per second average), giving near-real-time balances for dashboards; real-time rating only for customers with spending limits; a rating cut-off before the nightly billing run with a completeness check.

Validate with load tests at peak, measure end-to-end latency from event to rated result, and confirm the CC release supports both modes in the deployed landscape.

CC-16 — Pricing model selection

Flat rate (one price per unit) is simplest to maintain, test and explain. Tiered or graduated (each band priced separately) is fair, works event by event and has no price cliffs; it needs range tables and cumulative counters. Volume (all units at the tier reached) gives strong incentives, but creates price cliffs and needs period-end calculation. Time-dependent pricing (peak or off-peak, weekday or weekend) needs event timestamps in the right time zone. Contract-specific pricing (negotiated rates) should be parameters or mapping table rows keyed by contract or agreement, not separate charge plans per customer.

Maintainability: keep a small number of generic charge plans with parameters and reference tables, and avoid one charge plan per customer, which multiplies testing and upgrade effort.

Performance: each lookup and counter adds processing time; volume models at period end shift load to month-end; time-dependent pricing adds little cost.

Testing: tiered and volume need threshold tests; time-dependent needs boundary-time and time-zone tests; contract-specific needs a test per pattern, not per customer. Recommendation: tiered pricing with parameter-driven contract discounts covers most enterprise requirements with the least operational risk.

CC-17 — Price catalogue governance

The framework covers five controls. (1) Environments: catalogue changes are built in DEV, tested in QA with regression simulation, and promoted to PRD; no direct production catalogue edits outside the emergency process. (2) Approval workflow: pricing owner (commercial) approves the price, BRIM architect approves the technical design, finance approves revenue impact, and test lead signs off results.

(3) Version control: every catalogue export is stored with version number, change ticket, and date; changes use validity dates so history is preserved. (4) Emergency changes: allowed only for production defects with financial impact, approved by two named roles, applied in PRD with a mandatory retrofit to DEV and QA within an agreed time, and reviewed afterwards.

(5) Audit evidence: change ticket, before and after values, test results, approver names and timestamps, and a post-deployment check that sample events rate correctly.

Segregation of duties: the person who changes the catalogue cannot approve it, and production catalogue maintenance roles are limited. Report monthly on catalogue changes, emergency changes and pricing defects found in production.

CC-18 — Reconciling CC output with CI (800 missing items)

Compare at item level, not totals. (1) Extract the 800,000 charged items from CC for the period with their unique IDs, contract and amount. (2) Extract the 799,200 billable items

in CI with the source reference (charged item ID or external reference). (3) Match by ID; the unmatched 800 are the population.

Then classify each missing item by where it stopped: still in the CC output queue or file not sent; sent but rejected at CI upload (check the billable item upload error log or the error status in the billable item monitor — verify app for release); accepted into a status other than billable (for example, errored or excluded billable items); or mapped to a different billable item class not in the extract.

Recovery: fix the cause (missing contract account, invalid field mapping, unknown subprocess), then resend only the 800 items with their original IDs. CI's duplicate check on external IDs, where configured, protects against double upload; confirm before resending.

Controls: an automated daily count and amount reconciliation between CC and CI, alerting at any difference, and a rule that billing for a period does not close until the difference is zero or explained.

CC-19 — Performance drop after mapping table change

A 40% throughput drop after a mapping table redesign points to lookup cost. Measurement first: compare throughput (items per second), per-item processing time and CPU before and after, using the same test data in QA. Use CC performance statistics to see time per charge and per lookup.

Likely causes: the new table has many more rows or columns; lookups use non-key or wildcard matching that forces scans; the charge performs the lookup several times per item; the table no longer fits in the cache and is read from the database; or the change added chained lookups (mapping table result feeding another lookup).

Optimisation plan: reduce rows by grouping combinations with defaults; order input columns so the most selective come first; avoid repeated lookups by storing the result in a charge variable; split one large table into smaller tables by service category; review CC cache settings for reference data; and pre-enrich static attributes in CM so CC receives them on the chargeable item instead of looking them up.

Validate each change in a load test and roll out only when throughput returns to the baseline. Add a performance test gate to the catalogue governance process for future changes.

CC-20 — Complete charging FDD outline

The CC FDD should contain the following. (1) Scope: products, charge types and regions. (2) Logical design: charge plan structure diagram, charges per plan (recurring, one-time, usage, discount), counters and their reset rules. (3) Catalogue objects:

names, types, parameters (name, type, source system field, default), mapping and range tables with columns and sample rows.

(4) Sample inputs and outputs: a chargeable item example (fields, values) and the resulting charged item, with the calculation shown, for example 15,000 units → ₹25,500 under the CC-02 tiers. (5) Integration mapping: SOM contract attributes to CC parameters; CM chargeable item fields to charge inputs; CC output fields to CI billable item fields.

(6) Calculations: proration, rounding, currency and tier logic with worked examples. (7) Exception handling: rejected items, no-match lookups, missing contract, duplicate items, and the queue and owner for each. (8) Non-functional requirements: throughput, latency, retention. (9) Test cases: positive, negative, boundary, late usage, rerating, with expected amounts. (10) Assumptions, open points and sign-off. Every price in the FDD traces to an approved commercial source.

CM — Convergent Mediation

CM-01 — End-to-end mediation implementation

SAP Convergent Mediation by DigitalRoute converts raw usage from source platforms into clean chargeable items for SAP CC; it does not price. Design one workflow chain per source with shared downstream stages. Collection: file collectors (SFTP), API or message collectors (REST, Kafka) per platform, with checkpoints so restarts resume where they stopped. Parsing: a decoder per format. Validation: mandatory fields, data types, value ranges and timestamps. Normalisation: map each source into one canonical usage record. Enrichment: add contract ID, product and country from reference data synchronised from SOM or FI-CA. Transformation and aggregation where business rules allow. Delivery: send chargeable items to CC through the supported CC interface, with acknowledgements.

Error handling: invalid records go to an error store with a reason code, never discarded; correct and replay later. Monitoring: per-source counts in, valid, rejected and delivered; latency; backlog; alerts on gaps (a source with no files for an hour).

Controls: a completeness reconciliation per source per day — received = delivered + rejected + duplicates + pending. Version workflows, test with real anonymised samples from each platform, and confirm the CM version's supported connectors for CC.

CM-02 — Workflow design for JSON, XML, CSV and API sources

Use a decoder per input format feeding one canonical usage model. Canonical record fields: event_id, source_system, external_customer_or_service_id, service_type, quantity, unit_of_measure, event_start_time (UTC), event_end_time, location or country, and raw_reference for lineage.

Sample mapping rules: JSON $.usage.id → event_id; XML /Record/@uid → event_id; CSV column 1 → event_id with the source prefix added ("SRC3-" + value) so IDs are unique across sources. Quantities convert to the canonical unit (MB to GB with divide by 1,024, or seconds to minutes as agreed). Timestamps convert from source time zone to UTC with the source offset stored.

Schema validation: JSON Schema for JSON, XSD for XML, a column-count and type check for CSV, and payload validation at the API endpoint before acceptance. Records failing schema validation go to the error store with code and original payload.

Design tips: keep source-specific logic only in the first stage; everything after works on the canonical model, so adding a sixth source means one new decoder. Version the canonical schema and keep it backwards compatible. Test each decoder with valid, malformed, empty and oversized inputs.

CM-03 — Deduplication design

Build deduplication on a persistent key store. Key: source_system + source event ID; when the source has no reliable ID, a hash of stable fields (customer or service ID, start time, quantity, service type). Store each key with its first-seen timestamp in a persistent table or the CM duplicate-check function, and drop any record whose key already exists within the retention window.

Retention: set the window to cover the longest realistic resend delay plus margin, for example 45 to 90 days, and purge older keys on schedule. Records older than the window must be checked through a late-record rule rather than accepted blindly.

Restart-safe processing: the duplicate key insert and the record's forwarding must commit together, or the workflow must be idempotent; otherwise a crash after forwarding but before storing the key lets the resend through. Use CM transaction handling and checkpointing so a restarted batch reprocesses only uncommitted records.

Corrections are not duplicates: a resent record with a changed quantity needs a correction rule (reverse and replace) agreed with the source owner. Report duplicate counts per source daily, and keep the downstream CC duplicate check as a second line of defence.

CM-04 — 15-minute aggregation with audit preservation

Aggregate in CM by key (contract or service ID, service type, 15-minute window by event time) and send one chargeable item per key and window to CC. Preserve the original events in an archive store (CM storage or a data lake) with an aggregation ID that links each raw event to the aggregate it fed.

Window closing: close a window after its end plus a grace period (for example, 30 minutes) to catch slightly late events; events arriving after closure go into a correction aggregate for the same window with a correction flag, rather than reopening a delivered aggregate.

Lineage: aggregate record carries the aggregation ID, window start and end, count of events and sum of quantity; any customer dispute can be traced from the invoice line back to the raw events.

Correction handling: if raw events are later found invalid, create a negative correction aggregate and send it to CC for reversal, never edit the delivered one.

Reconciliation: sum of raw quantities per window equals the sum of aggregate plus correction quantities; the count of raw events equals the sum of event counts on aggregates. Confirm with the business which events must not be aggregated (individually priced or individually shown on invoices).

CM-05 — Error recovery for 500,000 failed records

Quarantine first: the 500,000 records (5% of 10 million) go to an error store with reason code "missing customer identifier", original payload, source file and line. Delivery of the 9.5 million valid records continues, so billing is not blocked by the errors.

Investigation: check whether the missing identifier is systematic (one source, one file, one time range, one new service type) or random. A sudden 5% failure usually means a source format change, a new product not yet in reference data, or a reference-data sync failure from SOM.

Correction: fix at the source when the data really is missing; fix the reference data or mapping when the identifier exists but enrichment failed. Do not hand-edit records except under approved procedure with an audit log.

Replay: reprocess corrected records through the normal workflow, including deduplication, so a record replayed twice cannot be charged twice. Prioritise records needed for the current billing cut-off.

Financial completeness controls: daily reconciliation (received 10,000,000 = delivered + in error + duplicates), an aging report on the error store, an estimate of the financial value held in errors, and a rule that the billing period does not close while material error volume remains unexplained.

CM-06 — Real-time architecture at 2,000 events per second

Use a streaming design: an API or message collector (for example, Kafka) receives events; CM workflows process in parallel instances partitioned by contract or service ID, so events for one customer stay in order. Processing per event: validate, deduplicate, enrich from cached reference data, then forward to CC.

Isolating downstream unavailability: place a persistent queue between CM processing and CC delivery. When CC is slow or down, CM keeps accepting and processing events into the queue; delivery resumes and drains the backlog when CC recovers. Apply back-pressure upstream only if the queue nears capacity. Size the queue for the longest outage you must survive: at 2,000 events per second, a two-hour outage means 14.4 million events.

Delivery guarantees: at-least-once delivery from CM plus duplicate detection in CC by chargeable item ID gives effectively exactly-once charging.

Capacity: test at 3,000 events per second (150% of target) and measure end-to-end latency, CPU and queue growth. Monitoring: events per second in and out, queue depth and age, error rate, CC response time, with alerts on queue age rather than only depth. Confirm supported CM collectors and CC interfaces for the deployed versions.

CM-07 — Data enrichment

Raw events have service ID and quantity; CC needs contract ID, product category, country and chargeable timestamp. Enrichment sources: a reference table mapping service or technical resource ID to provider contract ID and product, synchronised from SOM or FI-CA (through events on contract change or a scheduled extract); country from the contract or the service location; chargeable timestamp derived from the event time in UTC with the business time-zone rule.

Caching: hold reference data in CM in-memory lookup tables refreshed on change events or at a short interval, to avoid calling S/4HANA for each event. Track the cache refresh timestamp and alert if stale.

Effective-date validation: reference rows carry valid-from and valid-to dates; look up the mapping valid at the event time, not at processing time. This matters when a technical resource moves from one contract to another or when a contract changes product mid-month.

Missing mappings: send the event to the error store with code "no active contract for service ID on event date"; do not default to a generic contract. Re-run enrichment automatically once reference data updates.

Test: active, expired, future-dated and moved service IDs; cache refresh during load; and missing country.

CM-08 — Late usage events (three months)

Process by event time, not arrival time. Each record keeps its original consumption timestamp; enrichment looks up the contract valid at that time, and CC rates with the catalogue valid at that time.

Retention: CM must keep reference-data history and duplicate keys for at least the late-arrival window (here at least 90 days, preferably 120) or late events cannot be validated. Raw event archives follow audit retention rules.

Charging eligibility: define policy in writing. For example, events up to 90 days late are charged automatically; events older than that go to a review queue; events for contracts already terminated and final-invoiced need commercial approval. Check the commercial contract and local regulation, since some contracts limit backbilling.

Adjustment billing: late events are rated as new chargeable items with the original event date; CI bills them on the next invoice as a separate late-usage line referencing the original period, with tax per the supply date. Posted invoices are never changed.

Audit treatment: flag late events, report volume and value per source per month, and track the source platform's lateness as a supplier KPI. Agree with finance how late usage affects accruals and revenue for closed periods.

CM-09 — Throughput drop of 60% after a new enrichment stage

Instrument first: measure records per second and time per stage for each workflow, before and after the change, with the same input volume. The new stage is the prime suspect, but confirm with numbers.

Common causes: the enrichment calls an external system per record (a remote lookup to S/4HANA or a REST API) instead of using a cache; the lookup table is large and searched without an index or key; a synchronous call blocks workflow threads; cache refresh runs too often and locks the table; or the stage serialised processing that was previously parallel.

Optimisation: replace per-record remote calls with a local cache refreshed on change; batch lookups; index the lookup key; increase parallel workflow instances if CPU allows; move static enrichment (country from contract) to reference data rather than computed per event; and make external calls asynchronous with a timeout and fallback to the error store.

Capacity testing: run a load test at expected peak plus 50% in a production-like environment, compare against the baseline, and include cache-refresh during load. Roll out the optimised stage behind a version switch so you can return to the previous workflow. Add per-stage performance metrics to monitoring permanently.

CM-10 — Mediation design specification outline

The technical design contains these parts. (1) Overview: sources, volumes (average and peak per second), latency targets and target system (CC interface type and version). (2) Source schemas per platform with sample records; target schema (canonical record

and CC chargeable item fields with types and mandatory flags). (3) Validation rules: each rule with ID, condition and error code, for example V-01 missing event ID → E101.

(4) Transformations: field-by-field mapping, unit conversions, time-zone rules, aggregation keys and windows. (5) Enrichment: sources, cache refresh, effective dating. (6) Deduplication: key definition and retention. (7) Error codes catalogue: code, meaning, owner, correction action, replayable yes or no.

(8) Interface security: authentication for each collector (certificates, OAuth), encryption in transit, masking of personal data in logs, and service accounts with least privilege. (9) Monitoring: metrics, thresholds, alerts and dashboards. (10) Replay procedure: how to select, correct and reprocess records safely, with deduplication still active. (11) Test cases: valid, invalid, duplicate, late, boundary windows, volume and outage recovery, each with expected counts. (12) Operations runbook references, assumptions and sign-off.

CI — Convergent Invoicing

CI-01 — End-to-end CI configuration

CI sits inside FI-CA, so configure FI-CA first. (1) FI-CA basics: company code assignment, contract account categories, number ranges, main and subtransactions, account determination, payment terms and tax settings. (2) Business partner and contract account with billing-relevant data (billing cycle, invoicing category). (3) Billable items (IMG area under Contract Accounts Receivable and Payable → Convergent Invoicing → Basic Functions — verify path): define billable item classes and generate their interface and tables; define subprocesses; define billable item types and assign them to subprocess and class; define source transaction types. (4) Billing: billing types, billing process, selection and grouping variants, billing cycles, aggregation rules. (5) Invoicing: invoicing process, invoicing category, invoicing types, posting of billing documents, invoice output. (6) Integration: CC output mapped to the billable item class; tax through standard tax or external engine; transfer to general ledger.

Run-time transactions such as mass billing and mass invoicing run in the FI-CA mass activity framework (for example FKKBIX_MA and FKKINV_MA — verify). Test cases: upload a billable item, bill it, invoice it, check the FI-CA document and the account balance, then transfer to G/L. Evidence: screenshot or report at each stage with document numbers. Every IMG path and transaction must be confirmed for the client's S/4HANA release.

CI-02 — Billable item configuration

A billable item class defines the structure and database tables for a family of billable items (for example, usage from CC, recurring fees). Define the class with its interface components (basic data, tax, external references, custom fields), then generate the

tables. Subprocesses group billable items that are billed together under the same rules (for example, "telecom usage" versus "subscription fees"). Billable item types identify the specific charge kind (usage, recurring, one-time, discount) and are assigned to a subprocess and class. Selection criteria and grouping variants decide which items go into one billing document.

How CC charges become billable items: CC sends charged items (or consumption items in releases that use them) through the configured interface; CI receives them as raw billable items, then transforms them into billable items with the correct type, contract account and provider contract assignment.

Processing status: raw (received, not yet transformed), billable (ready), billed (in a billing document), and error or excluded states. Monitor these through the billable items monitor (app or transaction, for example FKKBIXBIT_MON — verify). Validation: every CC output item lands as exactly one billable item with matching amount, quantity, date and external ID; errors show a message that names the field.

CI-03 — Monthly billing configuration

Define one billing process for the subscription business with subprocesses for recurring, one-time and usage items (or one subprocess with different billable item types, depending on invoice layout needs). Billing selection: billing runs per billing cycle and contract account, selecting billable items up to the billing period end date (the cut-off). Grouping: grouping variants control how items form billing documents — per contract account and provider contract is common. Aggregation: usage items can be summarised into one billing line per product and period, while keeping the individual items for audit.

Scheduling: monthly mass billing run after the usage cut-off and after confirming CC delivery is complete for the period. Use parallel processing in the mass activity with intervals by contract account.

Billing document creation: each billing document holds the selected billable items, amounts and taxes basis; billable items move to billed status. Error recovery: items that fail (missing master data, inconsistent currency) stay billable or go to error with a message; fix and include them in a follow-up run. Billing runs are restartable: items already billed are not selected again. Reverse a wrong billing document with standard billing reversal before invoicing, and items return to billable status.

CI-04 — Consolidated invoicing for five contracts

Put the five subscription contracts' provider contracts under one contract account (or define a collective invoicing relationship if they belong to different contract accounts and the release supports it). Each contract is billed separately into billing documents;

invoicing then collects all billing documents for the contract account that are due and creates one invoicing document.

Invoicing configuration: invoicing process and invoicing type; invoicing category on the contract account; invoicing units if invoices must be split (for example, by department or by billing currency); selection of billing documents by date and status. Invoicing posts the amounts to FI-CA (one posting document with receivable line items per contract or main and subtransaction), calculates or finalises tax, and creates the print document for output.

Invoice output: Adobe Forms or the release's output management, showing a summary and a section per contract.

Reconciliation: sum of billing documents = invoice total before tax; invoice total including tax = FI-CA receivable; every billing document has exactly one invoicing document. Tests: all five contracts billed, one contract with zero charges, one contract in a different billing cycle (expected to wait or appear separately), and reversal of the consolidated invoice.

CI-05 — FI-CA integration

CI posts through FI-CA. Business partner and contract account hold the customer's receivables and payment, dunning and correspondence settings. Each invoice line maps to a main and subtransaction (for example, main transaction "subscription", subtransactions "recurring fee", "usage", "activation"). Billable item types map to main and subtransactions in CI configuration.

Account determination: main and subtransaction plus company code and other keys determine the receivables account (reconciliation account) and the revenue account. Tax codes determine tax accounts. Invoicing creates an FI-CA posting document with the receivable on the contract account and revenue and tax lines.

Receivables management: open items are cleared by payments (incoming payment processing, payment lot or direct debit run), by credit memo clearing, or by write-off. Dunning works on overdue items.

General ledger: FI-CA posts summary documents to the G/L through the transfer program (reconciliation keys group documents; close them and transfer daily). Reconciliation: FI-CA totals per reconciliation key equal G/L postings; contract account balances equal the receivables reconciliation account balance. Test the full chain: invoice, payment, clearing, dunning and G/L transfer, with document numbers at each step.

CI-06 — Consolidated B2B invoice for 100 subscriptions

One invoice per legal entity: a legal invoice cannot combine charges issued by different company codes, so "one consolidated invoice" means one per issuing legal entity, plus an optional non-legal summary statement across entities.

Structure: one business partner and one contract account per issuing legal entity; 100 provider contracts under it; each provider contract carries the department as an attribute (from SOM or as a billable item field). Billing aggregates 15,000 usage records into lines per contract, product and period; invoicing combines all billing documents for the period into one invoice.

Department breakdown: include the department field in billable items and billing lines, then print a department section on the invoice (summary per department, then contracts and usage per department). Provide a detailed usage annex as a separate file (CSV or PDF) rather than printing 15,000 lines.

Audit references: invoice line → billing document → billable items → CC charged items → CM aggregates → raw events, all linked by IDs. Performance: 15,000 items per month is small; the main design risk is invoice form length. Test with a full-size data set and confirm the customer's purchase order and department references appear as required.

CI-07 — Invoice correction for a ₹2 lakh overcharge

Two standard paths. (1) Full reversal: reverse the invoicing document (invoice reversal), which reverses the FI-CA posting; then reverse the billing document, returning billable items to billable; correct the billable items (or reverse and re-send from CC after rerating); rebill and reinvoice. This is clean but heavy, and changes the customer's invoice number. (2) Credit adjustment: keep the original invoice and create a credit of ₹2 lakh (plus related tax) through credit billable items from rerating or a manual credit with approval; it appears on the next invoice or as a separate credit memo. This is usually preferred once the customer has received the invoice.

Already-cleared receivable: if the customer paid, invoice reversal requires resetting clearing first, which creates an open payment. Prefer the credit memo; the credit becomes an open credit on the account that is offset against the next invoice or refunded on request.

Controls: approval by finance above a threshold, a reason code, tax adjustment on the same tax basis as the original (the credit note references the original invoice for GST or VAT), and a reconciliation showing original amount, credit and net correct amount. Root-cause the overcharge before closing.

CI-08 — Proration for a mid-month start

A ₹30,000 monthly subscription starting on the 16th can be prorated in three common ways. By actual days: in a 30-day month, 15 days (16th to 30th) → 30,000 × 15/30 = ₹15,000; in a 31-day month, 16 days → 30,000 × 16/31 = ₹15,483.87. By a fixed 30-day

month: 15/30 → ₹15,000 regardless. No proration: charge the full month, or start billing from the next cycle.

Ownership: proration should be calculated once, in one place. In a BRIM landscape, SAP CC normally calculates the recurring charge and its proration from the provider contract dates; CI then bills the prorated amount. If CI also prorates (for example, through billing period logic), the customer is prorated twice. Decide in the design which component owns it and switch the other off.

Rounding: round to 2 decimals at the charge level and apply the same rule in tests.

Billing-cycle alignment: decide whether the first invoice covers the 16th to month-end only, or the partial month plus the next full month (common for anniversary billing). Tests: start on the 1st, 16th, 31st and 29 February; mid-month upgrade; termination mid-month.

CI-09 — Tax determination for India, UAE and Germany

Tax is calculated at invoicing (or billing, depending on configuration) in FI-CA, either with standard S/4HANA tax procedures per country or with an external tax engine through the tax interface. Each country runs in its own company code with its own tax procedure.

India: GST (CGST plus SGST for intrastate, IGST for interstate), determined by supplier and customer GSTIN state codes, place of supply rules for services, and the service's SAC code. E-invoicing generates an IRN through the government portal for B2B invoices above the threshold.

UAE: VAT at the standard rate, with zero-rating or reverse charge for qualifying cross-border services; the tax registration number appears on the invoice.

Germany: VAT at the standard rate, reverse charge for B2B services to other EU businesses with valid VAT IDs, and e-invoicing obligations in B2B being phased in.

Data passed to tax determination: company code, customer tax ID and location, product tax classification, supply date and amount. Validate per legal entity: tax code determination for each scenario, invoice legal text, tax account postings, tax reporting extracts and credit note handling. Confirm current rates and e-invoicing rules with local tax advisers before go-live, because they change.

CI-10 — Weekly, monthly and quarterly charges for one customer

Decide first whether the customer wants one invoice or separate invoices per cycle. Separate invoices are simplest: give each charge type its own billing cycle and, if needed, its own contract account or invoicing unit; each cycle bills and invoices independently.

Consolidated option: bill each cycle on its own schedule (weekly billing creates billing documents every week), but invoice monthly, collecting all billing documents up to the invoice date. Quarterly charges then appear on the invoice of the quarter's billing month. This needs invoicing to select billing documents across cycles and the customer to accept weekly charges invoiced monthly.

Preventing misses and duplicates: billing selects billable items by billing period end date; items already billed cannot be selected again (status billed). Invoicing selects billing documents not yet invoiced. Duplicates arise mainly from wrong billable item creation (for example, CC generating the recurring charge twice); reconcile CC recurring charges against expected cycle counts.

Tests: a month with five weeks; a quarter end coinciding with month end; a change of cycle mid-quarter; and termination mid-cycle. Check the invoice shows each charge's service period clearly.

CI-11 — Missing billable items (1,500 of 100,000)

Trace the flow boundary by boundary with item-level IDs. (1) CC: confirm 100,000 items were actually transferred (not just rated), using the CC output or interface log. (2) Interface: check transfer files or messages for failures, partial loads or rejected batches. (3) CI raw billable items: check whether the 1,500 are present as raw items with errors (for example, contract account or provider contract not found, unknown billable item type, invalid currency). (4) CI billable items: check whether they are present but in error or excluded status, or under a different class or date range than the report used.

Matching: compare CC external IDs with CI billable item external references to list the exact 1,500 missing IDs and their characteristics; common patterns (one contract account range, one product, one day) point to the cause.

Recovery plan: fix the root cause (master data, mapping, configuration); then reprocess raw items in error, or resend only the missing items from CC with their original IDs. Confirm CI duplicate detection on external IDs before resending. After recovery, recount: CC 100,000 = CI billable 100,000. Include them in the current billing run if before cut-off; otherwise bill them next cycle with customer communication.

CI-12 — Billing stops at 70% during month-end

First action: check the mass activity status in the job and mass activity monitors — is it still running, cancelled, or waiting? Look at the application log for the run and the system logs for short dumps. Check intervals: which intervals finished, which are in process, which failed.

Lock analysis: check lock entries for contract accounts or billing tables; month-end billing often collides with other FI-CA mass runs (payments, dunning, invoicing) on the same accounts. Check work process availability and database load.

Processing status checks: count billable items already billed versus still billable for the period, so you know the exact remaining scope.

Restartability: FI-CA mass activities are designed to restart; restarting processes the remaining intervals, and items already billed are not selected again. Fix the cause before restarting (release locks from crashed processes after confirming they are orphaned, stop conflicting jobs, add work processes).

Safe rerun: restart the same run or start a new run for the same selection; never reset statuses manually in tables. After completion, reconcile expected contracts and billable items against billed, and only then start invoicing. For the future, set a month-end job calendar that separates conflicting mass activities.

CI-13 — Billing document exists but no invoice

Work through causes in order. (1) Was invoicing run for this contract account and date? Check the invoicing mass run selection (date, invoicing category, contract account range) and its log. (2) Eligibility: is the billing document marked for invoicing, and is its invoicing date reached? Billing documents may wait for a later invoicing date per the billing cycle. (3) Invoicing locks: contract account or billing document blocked for invoicing (invoicing lock on the contract account, a clarification case, or a billing document still in simulation status).

(4) Configuration: invoicing process and invoicing type assignment for the billing document's subprocess; missing assignment means the document is never selected. (5) Master data: contract account missing invoicing category, wrong company code, payment terms or tax data missing, causing invoicing errors visible in the log. (6) Posting errors: account determination or tax determination failures stop the invoice at posting.

Resolution sequence: read the invoicing log for the document, fix the error, rerun invoicing for that account in a single run first, then by mass run. Add a monitoring report of billing documents not invoiced after N days, which catches this proactively.

CI-14 — Duplicate invoice after a batch restart

First establish what actually exists. Compare the two invoices: different invoicing document numbers over the same billing documents means a real duplicate (should not occur in standard because billing documents get invoiced status); different billing documents over the same billable items means duplicate billing; different billable items with the same source reference means duplicates entered upstream (CC or upload resent). Also check whether one is a reversal document, a simulation or an output reprint rather than a second posting, since reprints and preliminary invoices are common false alarms.

Check FI-CA: are there two posting documents with receivables? Check the customer account balance.

Correction: if a genuine duplicate invoice posted, reverse the duplicate invoicing document with standard reversal (and its billing documents if they were duplicate), send a cancellation notice and fix the tax position. If the duplicate came from upstream items, reverse those billable items or credit them, and fix the interface.

Preventive controls: duplicate check on external IDs at billable item upload; restart of runs only through the mass activity restart, not by starting a parallel run over the same scope; and a post-run report flagging contract accounts with more than one invoice per period.

CI-15 — ₹15 crore CI versus FI-CA and G/L mismatch

Reconcile in layers with document-level matching. (1) CI to FI-CA: every invoicing document should have one FI-CA posting document. List invoicing documents for the period with amounts and their posting document numbers; find documents without postings (posting failed) and postings without invoicing documents (manual or other sources). (2) FI-CA to G/L: totals by reconciliation key and G/L account; find reconciliation keys not closed or not transferred.

Legitimate differences: timing (invoices created on the 31st, reconciliation key transferred on the 1st); tax (CI reports may show net while G/L shows gross plus tax); reversals and credit memos counted differently in each report; foreign currency translation; billing documents created but not yet invoiced (CI "billing value" is not posted until invoicing); and non-CI postings in the same accounts, such as payments, write-offs or manual adjustments.

Method: define the exact measure on each side (net or gross, document date or posting date, currency), extract at document level, match and list unmatched items. Every difference must be categorised: timing, tax, scope or error. Errors get root-caused. Make the reconciliation a monthly closing step with sign-off by finance.

CI-16 — High-volume billing (5 million items in 4 hours)

Five million items in four hours is about 350 items per second end to end. Design: aggregate in CM or CC where possible so CI receives fewer, already summarised billable items; spread billing of continuous charges through the month (bill weekly or daily into billing documents, invoice monthly) so month-end processes less.

Grouping strategy: billing documents per contract account and provider contract; aggregation rules that compress usage lines; avoid grouping that forces very large single documents.

Parallelisation: run mass billing with many parallel jobs and intervals sized to balance load (by contract account ranges with similar volume); avoid intervals dominated by one very large account.

Database considerations: billable item tables grow fast; plan archiving of billed items after the retention period, keep statistics current, and size HANA memory for the peak.

Performance baselines: measure items per second per job in QA with production-like volume, then calculate jobs needed with a safety margin, for example 30%.

Recovery: if the window is at risk, prioritise by invoicing deadline (largest customers first); restart failed intervals; have a documented decision point when invoicing proceeds for completed accounts and the rest follow next day.

CI-17 — Multilingual invoice form architecture

Use the output technology of the release: in S/4HANA, Adobe Forms through output management (or the FI-CA print workbench with application forms where that is the established setup). One form template with country variants is easier to maintain than five separate forms; texts come from translatable text modules, not hard-coded strings.

Data: the invoice print document from CI supplies header, lines, tax and references; customer-specific breakdowns (department, project, vessel) come from billable item fields carried into the print document. Tax disclosures differ by country: GST fields and IRN QR code for India, tax registration and VAT statements for UAE and Germany, reverse-charge wording where applicable.

Branding: logos and layout per legal entity, selected by company code. Language: from the business partner's correspondence language, with legally required local language where mandated.

PDF delivery: generate PDF, archive it (for audit and reprint), and send by email or portal; for e-invoicing countries, the legal invoice may be the structured XML file sent to the government platform, with the PDF as a visual copy. Tests: each country, each language, long invoices (page breaks), credit notes and reprints, and checks that the PDF and the FI-CA posting show identical totals.

CI-18 — Custom invoice field for PO and project references

Data source: the purchase-order reference and service project ID belong to the commercial contract, so capture them in SOM (contract or item level, using a custom field through key-user extensibility or a released extension), not in billing.

Mapping: pass the fields through the provider contract (or contract account where it is customer-level) and onto billable items. CI billable item classes allow customer-defined fields; add the fields to the class interface and regenerate the class (in DEV, transported). CC can also carry the reference on charged items if it is passed as a parameter.

Persistence: once on billable items, the fields are kept on billing document lines and are available to the invoice print document.

Invoice output: add the fields to the form's data structure and layout.

Upgrade compatibility: use released extension points (customer fields in the billable item class, key-user custom fields, BAdIs) instead of modifications; document every extension in the configuration workbook so upgrade impact analysis catches it.

Tests: field present on every invoice line, blank when not provided, value changes mid-period (which value prints?), and consolidated invoices with several POs. Alternative if only a header-level PO is needed: store it on the contract account and print it in the header.

CI-19 — Complete CI functional specification outline

The CI functional specification has these sections. (1) Scope, requirements and assumptions, including release and deployment. (2) Billable item acquisition: sources (CC, SD, external upload), classes, types, subprocesses, field mapping and error handling. (3) Billing: billing processes, cycles, selection and grouping, aggregation rules, cut-off dates and billing reversal. (4) Invoicing: invoicing processes and types, consolidation rules, invoicing units, tax determination, invoice output and archiving.

(5) FI-CA posting: main and subtransactions, account determination, reconciliation keys, G/L transfer and payment and dunning dependencies. (6) Exception handling: errors at each stage, monitors, owners, restart and reversal procedures. (7) Security: roles for upload, billing, invoicing, reversal, with segregation of duties; approval for credits above thresholds.

(8) Reports and reconciliation: CC to CI counts, CI to FI-CA, FI-CA to G/L, unbilled items and uninvoiced billing documents. (9) Technical dependencies: interfaces, batch schedule, volumes and performance targets, archiving. (10) Acceptance criteria per requirement with measurable expected results, and a traceability matrix to test cases. (11) Open points and sign-off.

CI-20 — Design review: bypassing CI billing

Creating FI-CA receivables directly from external usage events skips the controls CI exists to provide. Missing controls: no billable item store, so no single place to see unbilled, billed and reversed items; no grouping and aggregation; no standard billing reversal; no link from invoice line to source item; weak duplicate protection; and limited invoice presentation and tax handling at line level.

Trade-offs claimed by such designs (fewer components, faster build) are real only for very simple cases, such as one fixed charge per customer per month. For usage-based or multi-charge billing, the custom posting program becomes a homemade billing engine that must be maintained, tested at every upgrade and audited.

Business risks: revenue leakage when events fail silently; duplicate receivables after retries; inability to explain invoices in disputes; tax errors; audit findings on completeness; and expensive corrections, since reversing FI-CA postings without billing documents is manual.

Alternative: route events through CM and CC (or, for simple cases, upload billable items directly into CI through the standard billable item interface) and let CI bill and invoice. This keeps standard reversal, reconciliation and audit trail at modest extra cost. If CC is not needed, direct billable item upload into CI is a valid, lighter pattern.

IMPL — Cross-module implementation

IMPL-01 — Global greenfield BRIM implementation

Target architecture: Salesforce (opportunity, quote, order capture) → SAP Integration Suite → S/4HANA SOM (subscription contracts) → ODI → FI-CA provider contracts and SAP CC (charging) ← SAP CM (usage from platforms) → CC charged items → S/4HANA CI (billing, invoicing) → FI-CA (receivables, payments, dunning) → G/L, with SAP RAR for revenue if IFRS 15 applies.

Master data ownership: Salesforce owns prospects and opportunities; S/4HANA owns business partner, contract account and the subscription contract; CC owns the catalogue and charging state; CM owns usage processing rules. Products are mastered in S/4HANA and published to Salesforce.

Organisation: 12 company codes and sales organisations; contract accounts per legal entity; four invoice currencies set on contract accounts with CC priced in local currency.

Phases: (1) Discover and design, 8–10 weeks: requirements, solution design, catalogue design. (2) Build waves by country group. (3) SIT including volume tests for 500,000 subscribers. (4) Data migration rehearsals (at least three). (5) UAT. (6) Pilot country go-live, then remaining countries in two or three waves. Go-live plan: cutover runbook, billing parallel run against legacy for one cycle, hypercare of two billing cycles. Key risks: catalogue complexity, CRM integration quality and migration of in-flight contracts.

IMPL-02 — End-to-end order-to-cash trace

1. Salesforce order (external order ID) → SOM subscription order → subscription contract (contract ID). Checkpoint: contract active.
2. ODI distribution → FI-CA provider contract (same ID family) and CC provider contract with charge plan and parameters, linked to the subscriber account and contract account. Checkpoint: distribution status complete; CC contract exists.
3. Activation: CC triggers the one-time activation charge and starts the recurring charge on the billing cycle.

4. Usage: platforms send events (event IDs) → CM validates, deduplicates, enriches with contract ID, aggregates → chargeable items to CC. Checkpoint: CM counts received = delivered + rejected.
5. CC rates 15,000 units (charge plan tiers) → charged items with IDs. Checkpoint: rated quantity = delivered quantity.
6. CC → CI raw billable items → billable items (class, type, subprocess). Checkpoint: CC output count = CI billable item count.
7. Month-end billing → billing document; invoicing → invoicing document, invoice print and FI-CA posting document: debit receivable on the contract account, credit revenue and tax accounts. Checkpoint: invoice total = FI-CA receivable.
8. Payment → clearing; reconciliation key → G/L transfer.

Exception scenarios: distribution failure (no CC contract), usage without active contract (CM error store), rating rejection, billable item upload error, billing or invoicing errors, and payment failure leading to dunning. Each has a named monitor and owner.

IMPL-03 — Maritime industry BRIM architecture

Model the corporate customer (shipping company) as the business partner with a master agreement holding fleet-wide prices and discounts. Each vessel is a subscription contract item or a separate contract, with the vessel's IMO number as a key attribute; connectivity and digital services are subscription items with usage charges, equipment is a sales item, and maintenance is a recurring service item.

Vessel-level usage: terminals and network platforms send usage with terminal or SIM IDs; CM maps them to the vessel and then to the provider contract using effective-dated reference data, because terminals move between vessels.

Fleet changes: adding a vessel is a contract change (new item), selling a vessel is termination of that item; vessel transfers between fleets are termination plus new order, so history stays clean.

Discounts and global pricing: fleet-size discounts based on active vessel count, passed as a charge plan parameter or calculated through a shared counter; prices in USD as the maritime industry norm, with local currency where legal entities require it.

Consolidated billing: one invoice per customer and issuing legal entity per month, with a section per vessel. Key risks to design for: satellite usage arriving late (late-usage policy), time zones (use UTC), and shared fleet allowances (CC-07 patterns).

IMPL-04 — Simplified BRIM versus full BRIM

"BRIM Lite" is not a fixed SAP product; it means using only the components a business needs. For 25,000 contracts with recurring charges and limited usage, a simplified

architecture is SOM plus CI and FI-CA, with recurring charges generated without SAP CC, or a light CC charge plan setup, and usage either uploaded as billable items directly into CI (from a simple preprocessing step) or charged in CC without CM.

What can be omitted: CM, if usage volume is low and sources deliver clean data (a simple upload or Integration Suite flow can validate and load); possibly CC, if pricing is fixed recurring and simple usage can be priced before upload. Cannot be omitted: FI-CA and CI (billing and receivables), and SOM if contract lifecycle changes are frequent.

Required integrations shrink: no CM-to-CC flow; without CC, the price logic moves to the upload or to CI pricing, which is less flexible.

Operational consequences: fewer components to run and patch, simpler reconciliation, lower licence and infrastructure cost. Limits: complex usage pricing, real-time balances or high event volumes become hard later.

Scalability: keep the canonical billable item design compatible with CC output, so CC and CM can be added later without redesigning CI. Validate licensing implications with SAP.

IMPL-05 — Reconciliation framework across boundaries

The scenario has three losses: 0 units lost in CM (10,000 delivered), 500 units lost between CM and CC rating (10,000 → 9,500), and 200 units lost between CC and CI invoicing (9,500 → 9,300). Build a control at each boundary, matched by IDs, not totals.

Boundary	Measure	Expected	Actual	Gap
Source → CM	units received	10,000	10,000	0
CM → CC delivered	units delivered	10,000	10,000	0
CC rated	units rated	10,000	9,500	500
CC → CI billable	units as billable items	9,500	?	—
CI invoiced	units invoiced	9,500	9,300	200

For the 500: list chargeable items delivered but not charged; check CC rejections (no contract, invalid parameters, duplicate check, closed period). For the 200: split into not transferred, transferred but in error in CI, billable but not yet billed (later cut-off), and billed but not invoiced.

Attach financial values at each step using the rated price; for unrated items estimate with the expected tier price. Report daily per source and per customer, alert on gaps above tolerance, assign owners per boundary, and keep an aging report until every gap is resolved or written off with approval.

IMPL-06 — Downgrade not applied in CC

Symptoms: SOM shows the downgrade, CC keeps the old charge plan parameters, and CI invoices the old price.

Troubleshooting: (1) In SOM, confirm the change order, effective date and new time slice. (2) In the ODI monitor, check whether a distribution document was created for the change and its step status. (3) Compare the FI-CA provider contract and CC provider contract versions with SOM for the effective date. (4) If distribution succeeded but CC shows old values, check parameter mapping for the downgraded product and whether CC applied the change with the wrong validity date. (5) Check for concurrency with another change (SOM-18).

Correction: fix the cause and redistribute the change from SOM so CC gets the right charge plan from the downgrade date. Then rerate affected usage from that date and recalculate the prorated recurring charges; CC produces reversals and new charges. CI bills the difference as a credit on the next invoice (or reverse and reissue if the invoice is not yet sent, per policy). Inform the customer.

Preventive controls: alert on ODI errors; daily SOM-versus-CC contract parameter comparison for contracts changed in the last 48 hours; a billing pre-check that blocks billing for contracts with open distributions; and regression tests for each change type.

IMPL-07 — BRIM and RAR integration

Billing and revenue recognition are different processes: billing says what the customer owes and when; revenue recognition (IFRS 15 or ASC 606) says when revenue is earned. Responsibilities: SOM provides the contract, performance obligations and their prices, and amendments; CC provides usage-based amounts; CI provides invoiced amounts; FI-CA provides receivables and posts billed amounts to a deferred or contract liability or clearing account rather than directly to revenue when RAR is active; SAP RAR receives order (contract) and fulfilment and invoice data, allocates transaction price across performance obligations by standalone selling price, and posts revenue.

Flows: SOM sends contract and amendment data to RAR as order items; fulfilment events (time-based for subscriptions, usage-based for consumption) go to RAR; CI sends invoice data. RAR builds revenue contracts and calculates contract assets or liabilities.

Amendments: an upgrade mid-term is a contract modification; RAR treats it prospectively or with a cumulative catch-up depending on the accounting policy. Usage charges are often variable consideration recognised as consumed.

Design rule: never assume billing equals revenue. Test with finance: a 12-month contract with a free activation, an upgrade in month 4, and usage overage; check revenue, deferred revenue and contract asset balances each month.

IMPL-08 — BTP integration architecture

Use SAP Integration Suite as the central integration layer. Subscription activation: CRM order to SOM is usually asynchronous (accept, acknowledge, process, then send the result as an event), because contract creation and distribution take time. Charging data transfer: SOM to CC uses the standard S/4HANA–CC integration, not Integration Suite, unless the landscape requires routing. Usage processing: platforms send to CM directly (files or streams); Integration Suite only for API sources that need protocol mediation. Invoice notifications: S/4HANA events (invoice posted) published through SAP Event Mesh or Advanced Event Mesh to CRM and portals.

Synchronous versus asynchronous: synchronous for lookups needing an immediate answer (contract status, price simulation); asynchronous for anything that creates or changes data at volume.

Authentication: OAuth 2.0 client credentials or certificate-based mutual TLS; technical users with least privilege. Message persistence: Integration Suite data stores or JMS queues for guaranteed delivery. Retry: exponential back-off for technical errors, dead-letter queue after N attempts, and no automatic retry for business errors. Idempotency: an idempotency key (external order ID plus version) checked before creating anything. Monitoring: Integration Suite message monitoring plus correlation IDs carried end to end, alerting via Cloud ALM or the client's monitoring tool.

IMPL-09 — Business continuity plan for CC or CM outage

Objective: no usage lost, controlled delay and no duplicate charging. Ingestion buffers: CM persistent queues and file landing zones keep collecting while CC is down; if CM is down, sources must buffer or retain files for at least the RPO window. Confirm retention with every source owner.

Targets: example RTO of 4 hours for CC and CM; RPO of zero for usage data (nothing lost) because buffers retain it. Financial exposure: compute revenue at risk per hour of delay (unrated usage) and the impact on the billing deadline.

Failover assumptions: CC deployed with high availability (cluster nodes, database replication) and a disaster recovery site, if licensed and built; test failover twice a year.

During outage: CM keeps processing to the queue; CI billing for periods affected is postponed or run without the late usage, with usage billed next cycle; communicate to finance and key customers.

Restoration: bring CC up, verify catalogue and contract consistency, then drain the backlog at a controlled rate to avoid a second overload. Reconciliation: counts and amounts received during outage = delivered after restore; CC duplicate checks prevent double charging. Run a post-incident review and update the runbook.

IMPL-10 — Solution architecture approval pack

Prepare the design review pack with these sections. (1) Executive summary: scope, business outcomes, recommended architecture and key decisions needing approval. (2) Architecture diagrams: context diagram, component diagram (SOM, CC, CM, CI, FI-CA, RAR, CRM, Integration Suite) and data flow per process. (3) Interface catalogue: ID, source, target, pattern, frequency, volume, security, owner and error handling.

(4) Product and pricing decisions: catalogue structure, pricing models, proration and rounding rules, and where each is calculated. (5) Key configuration dependencies and the configuration workbook summary. (6) Security: roles, segregation of duties, data protection, encryption and audit logging. (7) Performance: volumes, targets and load-test plan.

(8) Test strategy: unit, SIT, volume, regression, UAT, parallel billing and reconciliation tests. (9) Risks and mitigations with owners. (10) Open decisions with options and a recommendation each. (11) Deviations from SAP standard and their justification.

Sign-off criteria: all critical requirements traced to design; no open high risks without mitigation; standard deviations approved; interface owners agreed; non-functional targets defined and testable; and finance agreement on reconciliation and revenue design. Record decisions with date and approver.

UPG — Upgrade and enhancement

UPG-01 — SAP CC 2023 to 2025 upgrade impact analysis

Start with release-path validation: confirm in the SAP CC 2025 upgrade guide and Product Availability Matrix that a direct upgrade from the installed 2023 support package is supported, and which minimum support package is required first. Check runtime prerequisites (Java version, operating system, database version) and whether the administration tooling (CC Cockpit components) must be upgraded together.

Compatibility analysis: the S/4HANA release and support package must be in the supported combination with CC 2025 for SOM–CC and CC–CI integration; check SAP Notes for the integration. Pricing catalogue assessment: export the catalogue, review deprecated features listed in the release notes, and simulate key charges after upgrade.

Integration impact: CM connectors to CC, the CC output interface to CI, and any custom clients calling CC APIs. Security: review changes to user roles and authentication in the new release and update technical users.

Test planning: regression suite (UPG-05) with parallel rating comparison old versus new. Downtime: SAP's guidance for this path requires stopping and restarting the complete Core Server, so plan an outage window with CM buffering usage. Rollback: full backup

of database and installation, documented restore procedure and a decision point in the cutover plan. Confirm every point against the official guide before scheduling.

UPG-02 — Impact of 150 custom enhancements

Build an inventory first: for each of the 150 enhancements record ID, module (SOM, CI, interfaces, output), technical object (BAdI, enhancement spot, custom program, CDS view, form, interface), business purpose, owner, usage frequency (from usage logs) and APIs or tables used.

Classify compatibility risk: green (uses released APIs or extension points with no changes in target release), amber (uses objects with changed behaviour or deprecation notices), red (modifications of SAP objects, direct table access to changed tables, or obsolete APIs). Use the S/4HANA custom code migration tools (ATC checks with the target release simplification database) and SAP's release notes for CC and CI.

Regression scope: each enhancement maps to test cases; red and amber items get dedicated tests, green items run in the general regression.

Remediation effort: estimate per item (hours) by category, with a contingency of 20–30% for red items. Technical debt: identify enhancements no longer used (retire them), duplicates of standard functionality now available (replace with standard) and modifications (rebuild with released extension points to follow clean-core principles).

Output: an impact register with risk, effort, owner and decision (keep, remediate, replace, retire), reviewed with the design authority.

UPG-03 — Zero or minimal downtime CC upgrade

First check the facts: the target release's upgrade guide states which steps require an outage. Where the documented path requires stopping the complete Core Server, true zero downtime for charging is not available through the standard procedure; do not promise it.

Realistic approach for near-continuous service: (1) Offline usage is buffered in CM during the outage and rated after restart; customers do not notice for postpaid billing. (2) Real-time scenarios (balance checks, spending limits) need a business rule during the window, for example allow service and rate later, with a risk limit. (3) Minimise the window: pre-stage installations, rehearse in a production-sized copy, and measure each step's duration. (4) Schedule at the lowest traffic time and away from month-end billing.

If the business truly needs active-active charging through upgrades, that is an architecture requirement (parallel CC landscapes with controlled switchover) with cost and data-consistency challenges; evaluate with SAP before committing.

Business continuity: communicate the window, monitor CM buffer growth, define a rollback trigger (for example, failing smoke tests within one hour of restart), and run a post-upgrade reconciliation of buffered versus rated items.

UPG-04 — New pricing enhancement: volume discounts, pooled allowances and promotions

Gap analysis per requirement. Volume discounts: standard CC counters and range tables support them (CC-06); gap is mostly design and testing. Pooled allowances: depends on CC release support for shared counters or sharing contracts (CC-07); validate, or design a parent-contract pattern. Contract-specific promotions (for example, three months 50% off): SOM handles the promotion as a contract attribute or discount item with validity dates; CC applies it through parameters or a discount charge.

Standard-first design. SOM: new product options and parameters for discount tier, pool membership and promotion code with validity. CC: new charge plan version with counters, a discount charge and range tables. CI: new billable item types for discounts and allowance usage so the invoice shows them as separate lines; check tax treatment of discounts. Reconciliation: report gross price, discounts and net per contract, and a pool usage report per group.

Impact on existing contracts: new charge plan version from a go-live date; migrate only contracts that receive the new features, through SOM changes.

Test cases: thresholds, pool exhaustion, promotion start and end, combination rules (can a promotion stack with a volume discount?), and late usage across a promotion end date.

UPG-05 — Automated regression strategy for a BRIM upgrade

Build a regression suite that compares old and new release outputs on the same input. Layers: (1) SOM: create, change (upgrade, downgrade, suspend, terminate), renew; verify contract status, time slices and ODI distribution success. (2) CC: replay a fixed set of chargeable items (golden data set covering every charge, tier boundary, counter reset and currency) and compare charged amounts line by line with the old release; any difference is a defect until explained.

(3) CM: replay sample files from each source; compare outputs, error codes and duplicate counts. (4) CI: upload billable items, run billing and invoicing; compare billing and invoice totals, tax and line structure. (5) FI-CA: compare posting documents, account determination and G/L transfer. (6) Reconciliation reports: run and compare.

Automation: SAP test tools (for example, the Test Automation Tool for S/4HANA or Tricentis where licensed) for UI and API flows; scripts for file replays and output comparison; a results dashboard showing pass, fail and differences.

Governance: freeze the golden data set and expected results before the upgrade; run regression after each support package as well. Pass criteria: 100% match on financial amounts, or each difference approved with a reason.

UPG-06 — Upgrade go-live decision

Recommendation: do not deploy to production yet. The evidence shows three unresolved risks, any one of which can affect revenue.

Finding	Risk	Threshold to proceed
2% rating differences	Wrong customer charges at scale	0% unexplained financial differences
12 intermittent integration failures	Lost or duplicated data	0 open critical integration defects
Month-end invoicing 30% slower	Missed billing window	Within agreed window with 20% margin

Action plan for the steering committee: (1) Rating: analyse every differing result, classify into expected (documented behaviour change, rounding change) or defect; fix defects and rerun the golden data set. (2) Integrations: trace each failure with correlation IDs; check new security settings, certificates, timeouts and API changes; rerun until stable over a full test cycle. (3) Performance: profile invoicing, update database statistics, review parallel job settings, apply SAP Notes for known performance issues; retest with production volume.

New decision gate: after fixes, a full regression and one simulated month-end run must meet all thresholds. Present a revised date with confidence level, the cost of delay, and the business impact of the current release support timeline, so the committee can decide on facts.

If the business needs a date commitment now, give a range rather than a single date.

PROD — Production support and architect challenges

PROD-01 — Priority 1 month-end incident

First 15 minutes: declare a P1, open a bridge with CM, CC, CI, Basis and finance leads, and name an incident manager and a communications owner. Freeze non-essential changes and jobs.

Investigation priorities, in order of money at risk: (1) Is usage lost or only delayed? Check CM queues and error stores: a backlog means delay, not loss. (2) Why is CC processing less? Check CC health (CPU, memory, threads, database), rejections and

the CM-to-CC connection. (3) What is the CI deadline exposure? Count contracts whose billing depends on the missing usage.

Containment options: increase CM-to-CC throughput if CC has capacity; otherwise throttle CM to a stable rate to stop timeouts and retries. Separate concerns: start CI billing for recurring charges and for contracts with complete usage, so most invoices finish before midnight. Agree with finance on the fallback: bill delayed usage in the next cycle as a separate line, or delay invoicing for a defined set of customers.

Communication: status every 30 minutes to stakeholders with facts, ETA and decisions needed.

Controlled recovery: drain the backlog, reconcile CM delivered versus CC rated versus CI billable before invoicing, and never rerun jobs without duplicate checks. Afterwards, hold a root-cause review within five working days.

PROD-02 — Financial defect after a transport

Contain first: stop the source of error by reverting or correcting the transported object (catalogue version, mapping, configuration) through emergency change, and pause billing for affected contracts if invoices are not yet issued.

Affected population: identify from the transport content which charges or rules changed and from what time; select charged items rated with the changed objects since import. Confirm by sample recalculation; expect fewer than 50,000 truly wrong if the defect only affects some cases.

Financial exposure: per customer, compute charged versus correct amount, total over- and under-charge, and split into unbilled, billed not yet sent, and invoiced.

Correction approach: unbilled items — reverse and rerate before billing. Invoiced — rerate to create reversals and correct charges, then bill differences as credits or debits on the next invoice; consider waiving undercharges below a threshold as a business decision.

Approval gates: finance approval of the correction amount, business approval of customer treatment, change board approval of the fix, and test evidence in QA on copied data.

Customer communication: clear message for over-charged customers with the credit amount; for under-charged customers, explain any additional charge before it appears. Prevention: add the case to regression and require rating comparison tests for catalogue transports.

PROD-03 — Duplicate charges only for amended subscriptions

Hypotheses: (1) Recurring charges are generated for both the old and new contract version over the same period because the change created an overlapping time slice. (2) Proration is calculated twice, once in CC and once in CI. (3) A change was distributed twice to CC (retry or concurrency), creating two contract versions. (4) Usage around the change date is rated on both versions. (5) Billing selects items from both the old and new provider contract.

Evidence gathering: take a set of affected and unaffected amended customers. For each, collect SOM change history, ODI distribution documents, CC contract versions with validity, charged items with dates, billable items, billing documents and invoice lines.

Data correlation: line up the duplicate charges with contract version validity; if duplicates fall exactly in the overlap between versions, hypothesis 1 or 3 is confirmed. If duplicates are recurring only, check proration ownership; if usage only, check rating by event date.

Root-cause validation: reproduce in QA with the same change type and dates; the duplicate must appear, and the fix must remove it.

Long-term prevention: fix the configuration or change process logic; add regression tests for each change type at different dates; add a billing pre-check that flags contracts with two recurring charges for the same period; and correct affected customers through credits.

PROD-04 — Security architecture

Data classification first: customer contract data, pricing agreements and payment-related data are confidential; personal data in usage events is regulated under privacy laws (GDPR in the EU, India's DPDP Act). Card data, if any, must stay in a PCI-compliant payment provider and only tokens should reach BRIM.

Access segregation: role-based access per module (SOM, CC catalogue, CM workflows, CI billing, FI-CA postings); segregation of duties, for example catalogue maintenance separate from catalogue approval, and billing reversal separate from credit approval; country-based restrictions so local teams see only their legal entities.

Service accounts: one technical user per interface, least privilege, no dialog logon, certificate or OAuth authentication, credentials in a secrets store and rotated regularly.

Encryption: TLS 1.2 or higher in transit for every interface; database and file encryption at rest; encrypted backups.

Audit logging: security audit log in S/4HANA, change logs for configuration and catalogue, CM workflow change history and access logs; send to a SIEM.

Data retention: legal retention for invoices and financial records (often 8–10 years by country), shorter retention for raw usage, with archiving and deletion schedules. Secure AI access: the AI agent uses read-only, scoped APIs, masked personal data, logged queries, and human approval for any write or financial action.

PROD-05 — Architectural disagreement on discount calculation

Evaluate both options against agreed criteria. Customising SAP CC: the discount lives where rating happens, so every charged item already includes it, the audit trail is in one place and late usage and rerating apply the discount automatically. Risks: complex logic in CC can hurt performance and make catalogue maintenance harder; truly custom code in CC (beyond configuration) raises upgrade cost.

External BTP service before rating: flexible, testable code, and easier to change by developers. Risks: a second pricing brain outside the charging system; rerating requires calling the service again with the historical rules and data; the service becomes part of the critical path for every event, adding latency and an availability dependency; the audit trail splits across systems.

Recommendation: implement the discount in CC using standard configuration (counters, range and mapping tables, discount charges), if it can be expressed that way, even if the configuration is more involved. Use BTP only to calculate slowly changing inputs offline (for example, a customer's discount tier recalculated monthly) and pass them to CC as contract parameters through SOM. This keeps one pricing engine, preserves rerating and audit, and keeps the BTP logic out of the real-time path.

Decide with a prototype: build the hardest discount case in CC configuration and measure performance before final sign-off.

PROD-06 — Delivery estimation for a six-month BRIM implementation

Six months end to end is aggressive; it is feasible only for limited scope: one or two countries, a small product catalogue, few integrations and clean data. State that assumption first.

Workstreams and indicative staffing at peak: architecture (1 lead architect); SOM (2 consultants); CC (2); CM (1–2); CI and FI-CA (2–3); integration (2 developers); data migration (2); testing (1 lead plus 3 testers); project management and change (2); client business and IT counterparts in each area. Roughly 18–22 people at peak.

Indicative phases: design 6 weeks; build 8–10 weeks; SIT and volume test 4 weeks; UAT 3 weeks; cutover and go-live 2 weeks; hypercare after. Data migration rehearsals run in parallel from build onwards.

Dependencies: SAP licences and landscape ready in week 1; CRM team availability; source systems' usage samples early; finance decisions on tax and revenue; and decision speed in the design authority.

Risks: catalogue complexity, migration of in-flight contracts and late usage samples. Deliverables: solution design, FDDs, configuration workbooks, interface specifications, test scripts and evidence, cutover runbook and support handbook. Present the estimate as a range with confidence level, and refine it after design.

BRIM-ARCH-001 — Flagship maritime scenario

The flagship needs a longer answer than 200 words because it combines design, calculation, defect diagnosis and go-live criteria. Assumption: the blueprint does not give the tiers above the free allowance, so this answer uses 10,001–50,000 units at ₹2 and above 50,000 at ₹1.50; replace with the real price list.

Solution design by module

- **SOM:** the shipping company is one business partner with a master agreement for fleet prices; each vessel is a subscription contract item keyed by IMO number. Change processes cover plan upgrade, downgrade, suspension (vessel in dry dock) and termination (vessel sold). ODI distributes provider contracts to FI-CA and CC.
- **CC:** one charge plan per vessel plan: recurring ₹50,000 monthly with day-based proration; one-time ₹1,00,000 activation; usage charge with a monthly counter per vessel (10,000 free, then tiers) reset each billing period. Currency per contract account.
- **CM:** collect usage from satellite and network platforms; deduplicate on source event ID; enrich terminal ID → vessel → provider contract with effective-dated reference data; process by event time (UTC).
- **CI and FI-CA:** billable item types for recurring, activation, usage and late usage; one contract account per customer and issuing legal entity; monthly billing and one consolidated invoice with a section per vessel; postings to main and subtransactions per charge type.

Sample calculations

Case	Calculation	Amount
New vessel, first month, 25,000 units	1,00,000 + 50,000 + (15,000 × ₹2)	₹1,80,000
Steady month, 8,000 units	50,000 + 0	₹50,000

Upgrade to ₹70,000 plan on day 16 of a 30-day month	50,000 × 15/30 + 70,000 × 15/30	₹60,000 recurring
Fleet base charge, 250 vessels	250 × 50,000	₹1,25,00,000

Open decision for the business: is the 10,000-unit allowance prorated when a plan changes mid-month, or kept per vessel per month? It changes usage amounts and must be fixed before CC build.

Retroactive billing for late usage (15–45 days)

Late events are rated with the contract and price valid on the event date. If the period is still open they join the current invoice; if invoiced, they appear as late-usage lines on the next invoice, referencing the service month. Because the counter for a closed month has been finalised, the design must state whether late units are priced at the tier the vessel reached that month (recommended, using the stored counter state) or at current-month tiers. Posted invoices are never changed.

Monitoring points and reconciliation

Daily controls at each boundary, matched by IDs: SOM active vessel contracts = FI-CA provider contracts = CC provider contracts; CM received = delivered + rejected + duplicates; CC rated = CI billable items; CI billed and invoiced totals = FI-CA receivables = G/L. Month-end close requires all gaps resolved or approved.

Defect: SOM activates 250 contracts, CC receives 248

Investigation sequence: (1) list the 250 contract IDs in SOM and the 248 in CC; identify the 2 missing; (2) check their ODI distribution documents and step status; (3) read error logs — typical causes are missing charge plan or parameter, contract account not ready, or a timeout; (4) check whether usage for those two vessels went to the CM error store (no active contract) and whether CI invoiced them without usage or without recurring charges.

Why CI invoices are wrong: the two vessels miss recurring and usage charges, or CM matched their terminals to an old contract. Fix the cause, restart the ODI steps, rerate the two vessels' usage from the error store, and bill the missing charges as a correction on the next invoice, or reverse and reissue the consolidated invoice if it has not been sent.

Controls against duplicate billing: replay only from the error store with original event IDs; CC duplicate check on chargeable item IDs; CI duplicate check on external IDs; and no manual creation of CC contracts.

Testing, migration and performance

Testing: SIT for each lifecycle event; month-end with 250 vessels including 20 mid-month changes and late usage; reconciliation tests; and a parallel billing run against the legacy system for one cycle. Migration: load active vessel contracts with their current plan and allowance state, set activation fee as already billed, and migrate open receivables in FI-CA. Performance: volumes are modest (250 vessels), so the main risk is late-arriving data rather than throughput; still test full-month usage replay in under the billing window.

Production approval criteria

- 100% contract consistency SOM = FI-CA = CC for all vessels.
- Zero unexplained differences in the parallel billing run; all differences categorised and signed off by finance.
- Late-usage, allowance proration and currency rules signed off by business.
- Reconciliation reports automated and owned.
- Runbook and recovery procedures tested, including the 248-of-250 scenario.
- No open critical or high defects; design authority and finance sign-off recorded.